

6

1) a) Push:-

This git command allows us to push a code from a local repository to a remote repository.

Syntax:- `git Push <url>`

Ex:- `git Push <https://giaks./git>`
`git push origin main.`

Merge:-

This command allows us to merge the code present in 2 different branches.

Syntax:- `git merge <Branch>name`

Ex:- `git merge Assist`

Commit:-

This command is used to save any changes made in the file, to its local repository.

Syntax:- `git commit <name> -m "Your message"`

Ex:- `git commit -m "New code added"`

Checkout:-

This command allows a user to change from the current branch to a different branch.

Syntax:- `git checkout <branchname>`

Ex:- `git checkout master.`

1)b) The git process and git flow are the main prerequisites.

- When a team creates a repository to store their code where everyone can access it and make changes.

- A certain process takes place in making a change in the local repository and then update it to the remote repository.

- Git Process and Flow

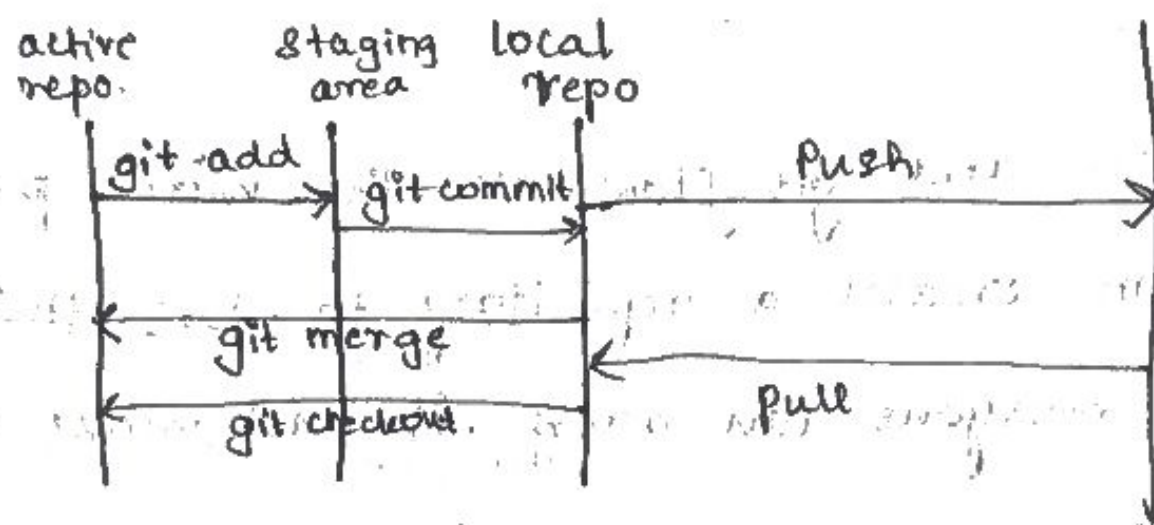
- When a developer₁ makes a change in the code and then pushes it to the main repository.

- And then developer₂ uses this code and makes more changes in the code and then uploads these changes to the remote repository.

- Then developer₁ can now pull this code and access the changes made by developer₂.

- The git flow works mainly in 2 parts the local repository and remote repository.

- The changes made in the local repository once ready to be deployed to be used by other team members or for testing etc can be sent to the remote repo.

local repositoryremote repository

- In the local repository we can add a file using `git add`.
- `git commit` is used to save any changes that are being made in the local repository.
- When the code is ready to be put to the remote repository we use `push` command to do so.
- Any code that needs to be taken from the remote repo to the local repo we use the `pull` command.
- And further in the local repository we use `merge` to merge different branches and `checkout` to switch between branches.
- This is how the git flow and process work.

- 2)a) The process to create a CI pipeline in Gitlab,
- Gitlab uses yml file to create a CI pipeline instead of a GUI.
 - To create a CI pipeline we can use the first use a docker image that allows us to understand the components that are required.
 - We used window.Net to create this CI pipeline.
 - .NET is a platform developed by windows which allows a user to create apps, website etc.
 - By using a yml file we first have to give the name of the project and follow the steps to create a pipeline:

docker configuration

Steps:- Build
test.

Element:- Build

ed build.

build configuration

\$ build configuration.

10

Step:- test.

Command:- cd test
app test.

Element:- Build
Test.

Test Configuration

\$ test configuration.

- Gitlab uses these methods to build a Continuous Integration pipeline.

2b) Github webhook is used to integrate applications such that when a change is made in the git repository, a notification related to it is automatically generated.

• To first make a webhook :-

① In github profile go to webhook and click on create webhook.

② In payload url :- Put the jenkins link followed by the webhook link.

③ Click on push only so that a notification arrives only in case of a push operation.

11 • In jenkins we can now get any ~~star~~ a notification in case there is a change made.

① We first check if the assigned URL is correct and it is connected to the correct repository.

② Any code that is pushed into the repository is now through this webhook going to notify us using the jenkins platform.

③ To make sure the integration is working correctly we can check by sending a input through the repository, if the jenkins platform pings that makes it integrated.

• A delivered ✓ sign is shown when the jenkins platform is working.

• payload URL <https://jenkins/.git/webhook.gi>